Technical Report: Movie Recommendation System

1. Introduction

In this project, we developed a movie recommendation system using the MovieLens 100K dataset. The goal was to create a model capable of predicting user ratings for movies and suggesting films that a user is likely to enjoy. The system is built using Python and various data science and machine learning libraries, with a focus on deep learning techniques.
The GitHub repository for the project contains the code, data handling scripts, model training routines, and instructions for running the system.

[Github repository]

---

2. Data

We used the MovieLens 100K dataset, which contains about 100,000 movie ratings from 943 users on 1,682 movies. We merged the movie metadata (movies.csv) and the user ratings (ratings.csv) using the movieId key to form a single dataset.

After merging, we refined the data by grouping ratings by userId and title, and calculating the mean rating per user-movie combination. Then, we applied label encoding to transform user IDs and movie titles into numeric representations suitable for use in a neural network.

The final dataset had the following format:

- **User ID** (encoded)
- **Movie Title** (encoded)
- **Normalized Rating** (scaled between 0 and 1)

We split the data into a training set (90%) and a test set (10%).

---

3. Model & Methods

We implemented a deep learning approach using the Keras and TensorFlow libraries. Our model is a neural collaborative filtering (NCF) architecture, combining user and movie embeddings followed by dense layers.

Key model components:

- Embedding layers for both users and movies (150 latent factors each)
- Dense hidden layers: [32, 16] units with ReLU activations
- Dropout (5%) applied after embeddings and each dense layer to prevent overfitting
- Output layer with sigmoid activation to output normalized ratings

Loss Function: Mean Squared Error (MSE)
Optimizer: Adam
Metrics: Mean Absolute Error (MAE)
Regularization: L2 regularization on embeddings
Training: Early stopping applied with patience of 3 epochs

---

4. Results & Evaluation

The model was trained for 10 epochs with early stopping based on validation loss. Performance was evaluated using Mean Squared Error and Mean Absolute Error on the test set.

Normalized ratings were used during training and inference to ensure consistency in prediction ranges. The model showed good generalization with low validation loss and limited overfitting, suggesting that the embedding-based approach effectively captured user preferences.

Further detailed metrics (like RMSE or Precision@k) can be added if evaluated later.

---

5. Contributions

- **Baptiste's Contributions:** Exploritory data analysis, exploring different datasets for better results, model building, status report, technical report.
- **Ikki's Contributions:** Data pre-processing, model building, setting up front- and back-end, set-up guide.
- **Online Resources:** Keras documentation, TensorFlow tutorials, and sample collaborative filtering models.
- **Use of GenAI:** ChatGPT was used to assist in designing the model architecture and debugging training issues.

---

6. Challenges & Future Work

Challenges:

- Overfitting during early experiments with deeper architectures
- Balancing model complexity and generalization due to relatively small dataset
- Proper rating normalization and encoding pipeline

Future Improvements:

- Implement a hybrid system combining content-based and collaborative filtering
- Add support for temporal data (timestamps)
- Evaluate with ranking metrics like Precision@k, Recall@k, and NDCG